

1 **Spherical Positional Encoding and Real-World Contextual Features for Transformer-**
2 **Based GPS Trajectory Classification**

3
4 **Alvin Wong***

5 Department of Computer Science
6 University of Florida, Gainesville, Florida, 32611
7 Email: alvinwong@ufl.edu

8
9 **Xiang Yan**

10 Department of Civil & Coastal Engineering
11 University of Florida, Gainesville, Florida, 32611
12 Email: xiangyan@ufl.edu

13
14 Word Count: 5,284 words + 4 tables (250 words per table) = 6,284 words

15
16
17 *Submitted August 1st, 2025*

18 *Alvin Wong

ABSTRACT

GPS data has emerged as a promising data source for human behavior analysis and transportation modeling. One key application of GPS data is GPS trajectory mode classification, which helps identify how individuals commute through urban areas. Understanding these transportation patterns is crucial, as transportation modes directly impact transportation infrastructure and urban mobility efficiency. While GPS trajectory mode classification has been well studied in the past, current approaches still face many limitations. One main limitation is how current approaches model and represent GPS trajectories. In this context, transformer models offer a promising solution due to their ability to handle long-term dependencies in sequential data that traditional models struggle with. However, standard transformer architectures are not designed for the unique characteristics of GPS trajectories, such as the spatial, temporal and geographical relations between points. This paper addresses this research gap by proposing a novel embedding strategy for transformer models that enhances their ability to model GPS trajectories. Specifically, we introduce the usage of spherical positional encoding to complement existing embedding layers, capturing a more accurate representation of GPS trajectories. In addition, we inform our model with real-world context by incorporating three new features: proximity to road, bus and walking routes. We evaluated our performance on three GPS trajectory datasets and conducted ablation tests to examine the performance compared to previous approaches. Our experimental results indicate that the inclusion of our novel embedding strategy and real-world contextual features provide consistent improvements compared to previous transformer-based models.

Keywords: GPS trajectory mode classification, Transformer, Spherical positional encoding

1 INTRODUCTION

2 Transportation mode choice is an important factor that urban planners must consider to maximize
3 the efficiency of transportation infrastructure. Transportation mode choice can impact traffic congestion,
4 travel times, and even the need for future transportation infrastructure [1,2]. Cities and transportation
5 institutions have mainly utilized interviews and household travel surveys to gather this information from
6 the population [3,4]. While this approach can potentially provide high quality data, travel surveys often
7 receive few responses, unreliable reporting and require a relatively large budget to conduct [3,4]. Another
8 data source, mobile GPS data, has become a promising alternative data source compared to travel surveys.
9 With the ubiquity of mobile phones and devices, mobile GPS data has seen a large rise in availability.
10 Therefore, GPS trajectory mode classification has emerged as a crucial task for taking advantage of
11 mobile GPS data to support data-driven and efficient urban planning.

12 Although GPS trajectory mode classification is a well-studied problem within existing literature,
13 current approaches still face many limitations [2,5,6]. A main limitation lies in the modeling and
14 representation of GPS trajectories, which vary across different approaches and often fail to capture an
15 accurate representation of a GPS trajectory. Early work relied on a set of hand-chosen features from GPS
16 trajectories, which were fed to traditional machine learning models for transportation mode classification
17 [6,7]. Models such as SVMs and Random Forests were commonly employed and produced strong results
18 despite their low complexity [6,7]. However, because these models cannot operate directly on the GPS
19 trajectories, they fail to capture the fine spatial, temporal and geographical relationships between points in
20 the GPS trajectory. Other literature has examined RNN based models due to their ability to directly
21 handle sequential data with temporal dependencies [2,8,9]. Models such as vanilla RNNs, LSTMs, and
22 GRUs are much more powerful than traditional machine learning models and are well designed for
23 handling sequential trajectory data but still face important limitations. One drawback is the default
24 assumption of fixed spacing between inputs, an assumption which does not hold for GPS trajectory data
25 [5,10]. In a GPS trajectory, the change in distance and time between successive points are variable. This
26 irregularity presents a major concern. If the distance and time intervals between points vary greatly or are
27 extremely high, it can introduce significant noise. Another limitation is their reduced ability to handle
28 long sequences. As sequence lengths increase, these models are prone to vanishing or exploding
29 gradients, a significant issue when some GPS trajectories can easily exceed 1,000 points. Therefore,
30 recent literature has pivoted to examine transformer models for GPS trajectory mode classification.

31 Transformer models can overcome both limitations through the usage of self-attention methods,
32 embedding layers and positional encoding [5,10]. Firstly, their self-attention mechanism allows
33 transformers to capture short and long-term dependencies across the entire trajectory regardless of the
34 trajectory's length [10]. Unlike RNN based models, transformers can process the entire sequence in
35 parallel, preventing the vanishing or exploding gradients issue that occurs with RNN based models.
36 Secondly, transformers can be modified to capture the variable time and distance differences between
37 successive points. While originally designed for natural language processing tasks, transformer models
38 can be adjusted for GPS trajectory mode classification using custom embedding layers and positional
39 encoding techniques [5,10]. GPS points, unlike words in text, are related to each other by time, distance,
40 geographic location and ordering. Therefore, the original transformer embedding layer and positional
41 encoding which operate on the ordering of words is insufficient in capturing a true representation of a
42 GPS trajectory and results in suboptimal performance. Prior work that examines the performance of
43 transformer models is limited. TMD-BERT was an early approach that utilized a pre-trained BERT model
44 and modeled the GPS trajectory as a text input [11]. As a result, it lacked spatial, temporal and
45 geographical encoding because it did not incorporate these features directly in the embedding layer or
46 positional encoding, leading to suboptimal performance. One work that partially addresses this limitation
47 is Trajformer [5], which introduces Continuous Positional Embedding (CPE) to inject information about
48 spatial and temporal relationships between trajectory points. However, CPE does not encode absolute
49 geographical position, limiting the model's ability to learn latent features such as bearing or turn angles
50 that are crucial for GPS trajectory classification.

To address this limitation in current transformer architecture, this paper proposes an improved embedding strategy that builds on previous literature by applying Spherical Positional Encoding (SPE) [12]. SPE is a variant of Rotary Position Embedding (RoPE) and embeds each GPS point as a position on the unit sphere, injecting information about the absolute geographical location of points [12]. SPE achieves this by applying rotation matrices to the query and key vectors, where the rotation matrices are dependent on the location of the point [12]. This encodes the location information directly into the self-attention mechanism, allowing the transformer to potentially learn the latent features reliant on location. In addition, we complement this with three new real world contextual features: proximity to road, bus and walking routes. These features add key information to the model but are rarely utilized in previous literature [5,6,8,9]. These features help to differentiate between transportation modes that follow similar trajectories but differ in environmental context. We conduct ablation testing on three GPS trajectory datasets to showcase the impact of our new embedding strategy and real-world contextual features. Our results demonstrate that SPE and the addition of our real-world contextual features consistently outperform other models and embedding strategies.

METHODS

The methodology section is broken down as follows. First, we provide a comprehensive overview of our approach and the justification behind it. Afterwards, we explain how our novel embedding strategy will be implemented for transformer models. Finally, we provide an overview of the datasets utilized and our preprocessing pipeline.

Overview of the Proposed Approach

Let each GPS trajectory represent a trip that an individual takes from a start to an end location. Formally, a trajectory can be defined as an ordered sequence of n points $T = \{p_1, p_2, p_3, \dots, p_n\}$ where each point $p_i \in R^n$ is represented as $p_i = (x_1^{(i)}, x_2^{(i)}, x_3^{(i)}, \dots, x_f^{(i)})$. Here, f is the total number of features and $x_j^{(i)}$ represents the value of the j^{th} feature for the i^{th} point in the trajectory. Different datasets may be created with various features such as instantaneous acceleration or angular velocity which require specialized sensors to measure [8]. However, in other literature, it is common to initially limit each point in the GPS trajectory to the latitude, longitude, and timestamp features [2,5,6,7,9]. In this case, $f = 3$ and $p_i = (\phi^{(i)}, \lambda^{(i)}, t^{(i)})$ where $\phi^{(i)}$ is latitude, $\lambda^{(i)}$ is longitude and $t^{(i)}$ is the timestamp of the i^{th} point. We also limit ourselves to only the latitude, longitude and timestamp. Given the above context, the problem of GPS trajectory mode classification can be defined as: given a trajectory T , is it possible to identify the transportation mode used throughout T ?

As discussed above, transformers are able of capturing more accurate representations of GPS trajectories. However, the main challenge in modifying a transformer model for GPS trajectory mode classification lies in appropriately modifying the embedding layer and positional encoding. To provide further context, let us first consider a transformer model designed for text classification. Given a sequence of text, the words in the text are first mapped to dense vector representations via an embedding layer. For simplicity, we can express the embedding as a linear transformation using one-hot encodings. The embedding is then defined as

$$X' = XE \quad (1)$$

where $X \in R^{n \times V}$ is the input sequence of n words with vocabulary size V , $E \in R^{V \times d}$ is the embedding matrix with hidden dimension d and $X' \in R^{n \times d}$ is the sequence of embedded vectors.

Afterwards, positional encodings are added to the embedded vectors. This is because multi-head self-attention is permutation equivariant and the ordering of the embedded vectors does not affect the content of the output [5,10]. Original transformers would employ techniques such as adding sinusoidal positional encodings to the embedded vectors, providing the model information about the word ordering [10]. The positional encoding step is defined as

$$Z = X' + P \quad (2)$$

where $P \in R^{n \times d}$ is the positional encodings to be added and $Z \in R^{n \times d}$ is the sequence of embedded vectors with positional encodings added. Afterwards, a learnable class token, known as the CLS, is prepended to the sequence of input vectors such that Z becomes

$$Z' = (z_{CLS}, z_1, \dots, z_n) \quad (3)$$

where $Z' \in R^{(n+1) \times d}$ is the final matrix to be fed to the transformer stack. The final matrix is sent through the stack of transformer blocks, each with a multi-head self-attention layer and a feedforward neural network. The multi-head self-attention layer computes three matrices from Z' .

$$Q = Z'W_Q \quad (4)$$

$$K = Z'W_K \quad (5)$$

$$V = Z'W_V \quad (6)$$

where $Q \in R^{(n+1) \times d_k}$ is the query matrix, $K \in R^{(n+1) \times d_k}$ is the key matrix, $V \in R^{(n+1) \times d_v}$ is the value matrix, d_k is the dimension of the query and key vectors, d_v is the dimension of the value vectors and $W_Q, W_K, W_V \in R^{d \times d_k}$ are the learned weight matrices for the queries, keys and values. Finally, the output of one head of multi-head self-attention is calculated as

$$Attention(Q, K, V) = softmax \frac{QK^T}{d_k} V \quad (7)$$

where $Attention(Q, K, V) \in R^{(n+1) \times d_v}$. In [10], they let $d_k = d_v = d / h$, where h is the number of heads used. It is evident from the formula for $Attention(Q, K, V)$, that rearranging the embedded vectors does not change the content of the output vectors, it only rearranges them in the same permutation as the embedded vectors. Without the positional encoding, self-attention would not be able to capture the semantic meaning that comes from word ordering because the content of the output would not change [10]. With positional information included, the CLS can meaningfully attend to all embedded vectors in the sequence [10]. This allows the model to capture both the meaning and position of each word in the text sequence, enabling the CLS to aggregate the proper information for classification [10]. Finally, the CLS would be fed to a final feedforward neural network to classify the text.

However, the embedding layer and positional encoding used for the above transformer model cannot directly capture the irregular nature of GPS trajectories [5]. Traditional transformer architectures use techniques explicitly designed for words which have uniform spacing [10]. Positional encodings, such as sinusoidal positional encoding, disregard the spatial, temporal and geographical information that is pivotal in GPS trajectory classification. GPS trajectory data differs significantly than text, carrying varying distance, time and location information between points. Consequently, the standard embedding layer and sinusoidal positional encoding are insufficient to capture these fine-grain details. Instead, we employ SPE on top of CPE to encode the ordering, time and distance differences and geographical location of points to reflect a more accurate representation of GPS trajectories. Without this specialized embedding strategy, the transformer model could lose critical context necessary for GPS trajectory mode classification.

Embedding Strategy

We will divide our embedding strategy into three steps as follows. First, we must enrich the raw trajectory by adding derived and contextual real-world features to our raw trajectory. Secondly, we use CPE as described above to embed our enriched trajectory. Finally, we employ SPE to encode the absolute geographical location of the points in the embedded trajectory. After embedding, the embedded vectors are passed through the standard blocks of multi-head self-attention and feedforward neural network layers, followed by a final feed-forward neural network to classify the transportation mode of the GPS trajectory.

Trajectory Enrichment

Recall that we start with trajectory $T = \{p_1, p_2, p_3, \dots, p_n\}$ where $p_i = (\phi^{(i)}, \lambda^{(i)}, t^{(i)})$. To add additional context to our trajectory, we add velocity and acceleration, two commonly derived features in previous literature [5,6,7]. We calculate the velocity at each index to be

$$v^{(i)} = \begin{cases} 0 & \text{if } i = 1 \\ \frac{\text{Haversine}(\phi^{(i)}, \lambda^{(i)}, \phi^{(i-1)}, \lambda^{(i-1)})}{t^{(i)} - t^{(i-1)}} & \text{otherwise} \end{cases} \quad (8)$$

where the Haversine function computes the great-circle distance between the two points. Then we calculate the acceleration at each index to be

$$a^{(i)} = \begin{cases} 0 & \text{if } i = 1 \\ \frac{v^{(i)} - v^{(i-1)}}{t^{(i)} - t^{(i-1)}} & \text{otherwise} \end{cases} \quad (9)$$

Afterwards, we calculate the proximity to road, bus and walking paths. Using the global map data from OpenStreetMap (OSM), we can extract a set of all points that correspond to either road, bus or walking paths. For each point p_i we calculate its minimum geographic distance to each path type, giving us $r^{(i)}, b^{(i)}, w^{(i)}$, representing the closest distance to a road, bus and walking path respectively. After calculating both the derived and real-world contextual features, we update our trajectory, with each point now being represented as $p_i = (\phi^{(i)}, \lambda^{(i)}, t^{(i)}, v^{(i)}, a^{(i)}, r^{(i)}, b^{(i)}, w^{(i)}) \in \mathbb{R}^8$.

Continuous Point Embedding

Next, we embed the updated trajectory using CPE as outlined in [5]. First, let us define $h(p_i) \in \mathbb{R}^d$ to be the point embedding produced by a learned multi-layer perceptron, which maps the raw feature vector $p_i \in \mathbb{R}^8$ to a latent embedding space. Additionally, let $g(\Delta t_{ij}, \Delta d_{ij}) \in \mathbb{R}$ be the scalar output of another learned multi-layer perceptron where Δt_{ij} and Δd_{ij} represent the difference in time and distance respectively between points p_i and p_j . Finally, let $N(p_i) \subset T$ be the neighborhood for each point p_i where $N(p_i) = \{p_j \in T \mid |i - j| \leq \lfloor k / 2 \rfloor\}$ and k is the odd-valued window size for our neighborhood. Then we can calculate the embedding of one GPS point as

$$CPE(p_i) = \frac{1}{|N(p_i)|} \sum_{p_j \in N(p_i)} (\Delta t_{ij}, \Delta d_{ij}) h(p_j) \quad (10)$$

where $CPE(p_i) \in \mathbb{R}^d$. Embedding the trajectory in this way captures context based on the time and distance difference between two points. When applied to the neighborhood of p_i , the function $g(\Delta t_{ij}, \Delta d_{ij})$ can be interpreted as generating a dynamic 1D convolution kernel over this neighborhood, where the weights of the kernel depend on the time and distance differences. Therefore, CPE is able to model the variable time and distance differences between successive points in a GPS trajectory. However, as noted before, it does not encode information about the absolute location of the points.

Spherical Positional Encoding

After embedding the updated trajectory to embedded vectors of dimension $\mathbb{R}^d$, we employ SPE to inject information about the absolute location of the points. In [12], it defines the rotation matrix as

$$R_e' = \begin{pmatrix} \cos(\phi) & -\cos(\lambda)\sin(\phi) & \sin(\lambda)\sin(\phi) \\ \sin(\phi) & \cos(\lambda)\cos(\phi) & -\sin(\lambda)\cos(\phi) \\ 0 & \sin(\lambda) & \cos(\lambda) \end{pmatrix} \quad (11)$$

This rotation matrix rotates a 3D vector by ϕ around the x-axis, and then by λ around the z-axis. To apply this rotation matrix to an embedded vector of a higher dimension, they block diagonalize the embedding matrix across the embedding dimension d . They define this new rotation matrix as

$$R_{\lambda, \phi}^{(d_k)} = \begin{pmatrix} \cos(\phi) & -\cos(\lambda)\sin(\phi) & \sin(\lambda)\sin(\phi) & \dots & 0 & 0 & 0 \\ \sin(\phi) & \cos(\lambda)\cos(\phi) & -\sin(\lambda)\cos(\phi) & \dots & 0 & 0 & 0 \\ 0 & \sin(\lambda) & \cos(\lambda) & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & \cos(\phi) & -\cos(\lambda)\sin(\phi) & \sin(\lambda)\sin(\phi) \\ 0 & 0 & 0 & \dots & \sin(\phi) & \cos(\lambda)\cos(\phi) & -\sin(\lambda)\cos(\phi) \\ 0 & 0 & 0 & \dots & 0 & \sin(\lambda) & \cos(\lambda) \end{pmatrix} \quad (12)$$

1 Using this rotation matrix, we rotate the query and key vectors before calculating $Attention(Q, K, V)$. Let
 2 $q_i \in Q$ and $k_i \in K$. To rotate the query and key vectors we perform matrix multiplication using $R_{\lambda, \phi}^{(d_k)}$. We
 3 obtain $q_i' = R_{\lambda, \phi}^{(d_k)} q_i$ and $k_i' = R_{\lambda, \phi}^{(d_k)} k_i$, where q_i' and k_i' are the rotated query and key vectors
 4 respectively for the i^{th} embedded vector. This rotation is applied to all $q_i \in Q$ and $k_i \in K$, giving us Q' and
 5 K' , which are the rotated query and key matrices respectively. The rotated query and key matrices are then
 6 used directly in the attention calculation as

$$Attention(Q', K', V) = softmax \frac{Q' K'^T}{d_k} V \quad (13)$$

7 This method encodes the absolute global location of the point p_i through the rotation of the query and key
 8 vectors. This is because the rotation matrix is determined by the latitude and longitude of the point p_i ,
 9 encoding the location of the point in the rotation [12]. Most importantly, SPE complements CPE by
 10 encoding absolute geographic location with CPE encoding relative point position into the embedded
 11 vectors. Together, they allow the model to capture both types of information which are key in
 12 distinguishing different transportation modes. We provide a diagram of our embedding strategy below.
 13

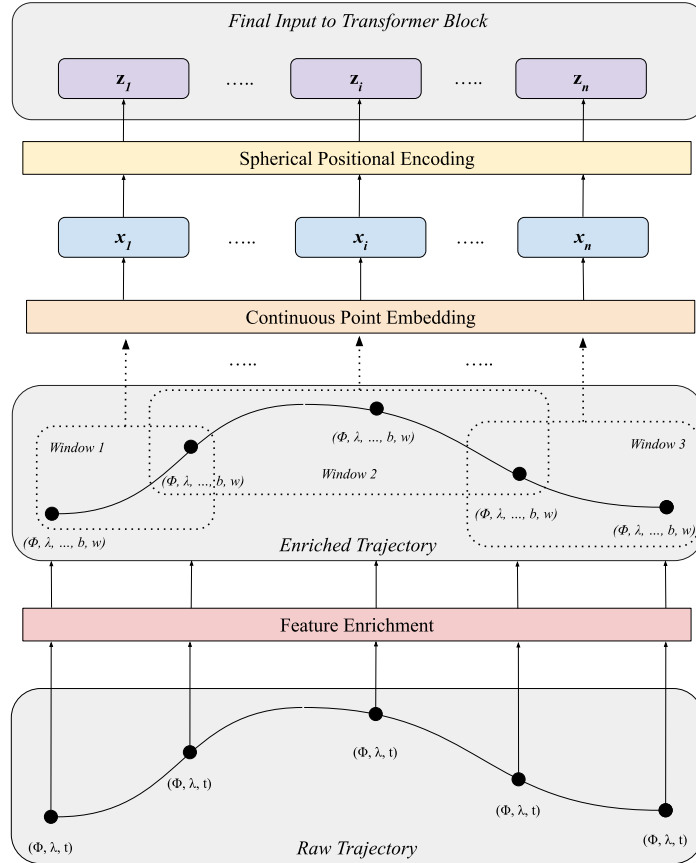


Figure 1 Diagram of Embedding Strategy

Data & Preprocessing

We utilized three GPS trajectory datasets to evaluate the performance of our new embedding strategy and real-world contextual features. To ensure consistency, all trajectory datasets were cleaned and processed in the same format. All trajectories had points ordered by timestamp, with points exhibiting duplicate timestamps or locations removed. Trajectories with fewer than 5 points were filtered out to ensure sufficient information was present for classification. Additionally, trips with unrealistic speeds or accelerations were also removed to reduce noise. Finally, all features were normalized to reduce feature magnitude imbalances. This pipeline provided clean data suitable for model training and evaluation. We now briefly describe the datasets we utilized below.

Geolife Dataset

The Geolife dataset [13] is a free GPS trajectory dataset collected for the GeoLife project. It contains trajectories from a broad range of outdoor activities with most trajectories being located in or near Beijing, China. The trajectories were recorded by different GPS loggers and mobile GPS devices. Although the dataset includes a wide range of transportation mode labels, we focused exclusively on trajectories labeled as car, bus, bike, or walk.

North Florida Dataset

The North Florida trajectory dataset is built off the dataset obtained and used in [14], containing raw GPS data from mobile devices in the northern half of Florida. They obtained GPS trajectories using the methods of home and stay point analysis. The home location was inferred by segmenting the area into 20-meter grids and designating the home location as the grid with the highest number of nighttime GPS points. Stay points were inferred by examining times outside of nighttime with minimal movement. They utilized the Trackintel package to detect stay points to form GPS trajectories representing distinct trips. Afterwards, trips were manually labeled by projecting the GPS trajectories onto a map of the area and examining their speed, acceleration, time of day, etc. This dataset was also augmented with car GPS trajectories from Open Street Map and bus GPS trajectories from real-time GTFS data. After the data processing and augmentation, ~2,300 labelled trips were extracted spanning across the labels of car, bus and walking.

OpenStreetMap Dataset

The OpenStreetMap [15] dataset was obtained from public GPS trajectories hosted on the OpenStreetMap website. Trajectories can be labeled with any tags but often include tags that describe their transportation mode. We extracted trips with labels which only corresponded to a single transportation mode. Because OpenStreetMap is used globally, our dataset also contains trajectories from across the globe. This sets it as a stark contrast to many other GPS trajectory datasets which only include trajectories from a single geographical location. To the best of our knowledge, we are the first paper to examine a global GPS trajectory dataset for GPS trajectory mode classification.

TABLE 1 Dataset Statistics

Dataset	Geolife	OpenStreetMap	North Florida
Number of trajectories	8,777	44,212	2,286
Total distance (m)	63,635,412	558,532,992	18,363,964
Total duration (s)	20,543,748	166,392,688	3,224,604
Length range (points)	5 ~ 9254	5 ~ 19839	5 ~ 194
Class ratios	Bike: 17.899% Bus: 21.260% Car: 14.914% Walk: 45.927%	Bike: 22.743% Bus: 15.414% Car: 35.244% Walk: 26.599%	Bus: 43.745% Car: 43.745% Walk: 12.511%

Feature Extraction for Traditional Models

Because traditional machine learning models cannot directly process GPS trajectory data, we extracted a set of spatial and temporal features to represent the trajectory. These derived features are consistent with those used in previous literature [6,7]. Additionally, we incorporate our real-world contextual features. An overview of the derived features is provided in the table below.

TABLE 2 Derived Features for Traditional Models

Features	Number of Variables
Origin-destination straight-line distance	1
Cumulative trip distance	1
Velocity percentiles (0th, 25th, 50th, 75th, 100th)	5
Acceleration percentiles (0th, 25th, 50th, 75th, 100th)	5
Proximity to road path percentiles (0th, 25th, 50th, 75th, 100th)	5
Proximity to bus path percentiles (0th, 25th, 50th, 75th, 100th)	5
Proximity to walking path percentiles (0th, 25th, 50th, 75th, 100th)	5

Model Training

After data collection, we trained multiple models for ablation testing to evaluate their performance across the three GPS trajectory datasets. To ensure a fair comparison across the different model types, we attempted to standardize the hyperparameter and training process when possible.

All transformer-based models shared the same hyperparameters as Trajformer [5], such that $L = 2$, $d = 64$, $N_h = 4$, where L is the number of transformer blocks, d is the hidden dimension and N_h is the number of heads. For our RNN-based models, we used a single hidden layer with a hidden size of 64. The traditional machine learning models employed grid search over a small set of hyper parameters to select the best set of hyperparameters based on the validation accuracy. To standardize the training process, RNN and transformer-based models were trained for 100 epochs with a batch size of 128, with early stopping applied based on validation accuracy with a patience of 10. The 80/10/10 train, validation, and test splits were produced using random seeds to ensure that each model was evaluated on the same data. The best model was then evaluated on the held-out test set.

Additionally, to prevent long training times and memory limit errors for our RNN and transformer-based models, we split long trajectories into sub-trajectories of 2,500 points. This not only reduced training time but also stabilized learning. Trajectories smaller than 2,500 points were padded to reach this limit.

RESULTS

We now present the results of our ablation tests below. The first table provides the overall test accuracy for every model tested. The provided subheaders “No context” and “Context” refer to the usage of the real-world contextual features during model training and testing. The second table provides the precision, recall and F1 score of each transformer model. For succinctness, we focus only on the transformer models trained with real-world contextual features. The subheaders of “Prec”, “Rec”, and “F1” correspond to precision, recall and F1 score respectively. Finally, we provide a figure to showcase the per-class F1 score of each transformer model. The x-axis subheaders “GEO”, “OSM”, “NF”, correspond to the GeoLife, OpenStreetMap and North Florida datasets respectively.

Accuracy

Figure 2 presents the accuracy results from the ablation experiments across the GeoLife, OpenStreetMap, and North Florida datasets. The results compare models trained with and without contextual features. From the figure, we make several observations discussed in the following paragraphs.

Our first observation is that adding contextual features improves performance across nearly all models and datasets, but the gains are dependent on the architecture of the model and the dataset used for training. When averaged across all models, adding real-world contextual features yields a +1.166-pp improvement for the GeoLife dataset, a +0.883-pp improvement for the OpenStreetMap dataset and a +0.927-pp improvement for the North Florida dataset. The gain on each dataset is relatively similar, suggesting that different geographical areas benefit consistently from the addition of contextual features. Additionally, models that obtain the greatest performance improvements from the addition of contextual features are the feature-based and recurrent-based models, rather than the transformed-based models. While the overall improvements are modest, they demonstrate that contextual features contributes measurable and consistent improvements to accuracy across diverse settings. Our second observation is transformer-based models exceed feature-based and recurrent-based models in almost all settings. For example, averaged across all datasets, the Base Transformer alone obtains an average accuracy of 92.546%. This exceeds the best feature-based model, RF, with an average accuracy of 91.842% and the best recurrent-based model, GRU, with an average accuracy of 91.709%. Our final observation is that our novel proposed architecture is the most consistently accurate. With contextual features, our proposed architecture achieves the best performance across the GeoLife, OpenStreetMap and North Florida datasets, achieving an accuracy of 89.955%, 93.199% and 96.070%, respectively. These results highlight the potential of combining both contextual and positional embeddings, which enable the model to better capture spatial temporal dependencies inherent in GPS trajectories. The strong performance across all datasets suggests that our architecture generalizes well across different geographical areas. Overall, the ablation study confirms that contextual information enhances model performance, transformer-based architectures deliver the highest baseline performance, and our proposed model achieves the best overall accuracy across our three datasets.

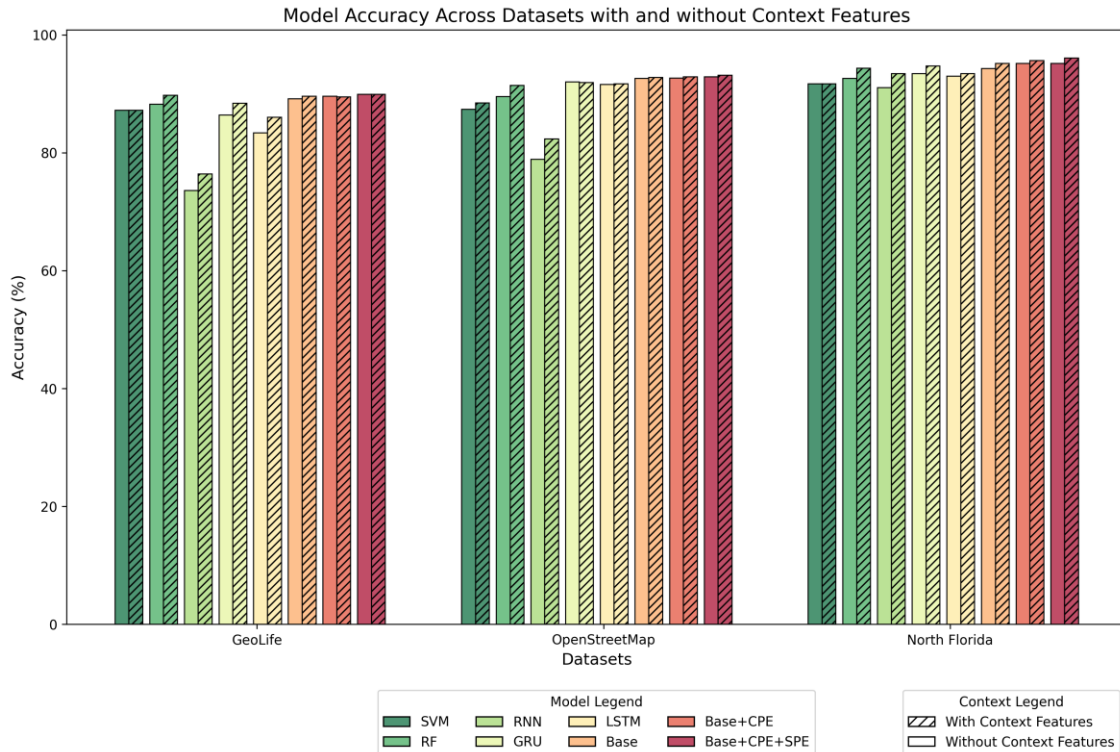


Figure 2 Graph of Model Accuracy Across Datasets with and without Context Features

Recall, Precision and F1 Scores

In addition to overall accuracy, we examine macro precision, recall, and F1 scores to provide a more comprehensive assessment of model performance. For brevity, we only consider the models trained with the addition of contextual features. Table 3 presents the macro precision, recall, and F1 results from the ablation experiments across the GeoLife, OpenStreetMap, and North Florida datasets.

From the table, we make two main observations. Our first observation is that precision and recall are consistently well balanced across almost all models and datasets, with differences typically within only a few percentage points. For example, when averaged across all models, the precision and recall differ by less than about 2 percentage points on the GeoLife dataset, less than 1 percentage point on the OpenStreetMap dataset, and less than 2 percentage points on the North Florida dataset. The corresponding precision and recall scores also closely follow the accuracy trends. This balance suggests that our models do not sacrifice recall for precision or vice versa, and that the precision recall tradeoff is managed well across architectures and geographic regions. Our second observation is that our proposed Base + CPE + SPE architecture achieves the strongest overall combination of precision, recall, and F1 score across all three datasets. Once again, these results suggest that our proposed architecture offers an improvement in precision and recall when compared to different models across diverse geographical settings.

TABLE 3 Precision, Recall and F1 Results from Ablation Testing

Model	GeoLife			OpenStreetMap			North Florida		
	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1
SVM	84.2%	80.7%	82.0%	87.4%	86.0%	86.6%	94.6%	95.7%	94.9%
RF	88.5%	86.8%	87.6%	90.9%	90.1%	90.4%	95.2%	94.9%	94.9%
RNN	75.2%	69.1%	70.8%	81.7%	81.4%	81.5%	92.4%	95.0%	93.4%
LSTM	85.0%	82.8%	83.5%	91.6%	91.2%	91.4%	93.2%	94.3%	93.6%
GRU	87.2%	86.1%	86.4%	91.6%	91.7%	91.6%	94.9%	96.0%	95.3%
Base transformer	87.6%	86.8%	87.1%	92.5%	92.8%	92.6%	94.1%	96.3%	95.1%
Base + CPE (Trajformer)	87.2%	87.3%	87.0%	92.5%	92.9%	92.7%	94.4%	95.8%	95.0%
Base + CPE + SPE	87.6%	87.8%	87.7%	92.9%	93.2%	93.0%	94.7%	96.1%	95.4%

Per-Class Accuracy

We now examine the per-class accuracy of our models across our datasets. For brevity, we will only examine the transformer-based models trained with the usage of contextual features. In Figure 3, we can see the accuracy of the models across all datasets and transportation modes.

From the figure, we make two observations. Our first observation is that per-class accuracy is mixed across the three transformer-based models, especially on the GeoLife dataset. For GeoLife, different variants obtain the highest accuracy for different transportation modes, and the gaps between models can vary by several percentage points, suggesting that no single transformer configuration consistently performs the best across classes on this dataset. Our second observation is that, despite this variability, our proposed architecture still yield modest but consistent improvements on the OpenStreetMap and North Florida datasets. On these datasets, the Base + CPE + SPE model generally matches or slightly exceeds the accuracy of the other transformer variants for most classes, particularly for the bus and car modes, while remaining competitive for the remaining classes. Nevertheless, the observed variability across classes and datasets indicates that further experimentation is needed and may also suggest that the combined effect of CPE and SPE on per-class performance is more limited than our aggregate results alone would imply.

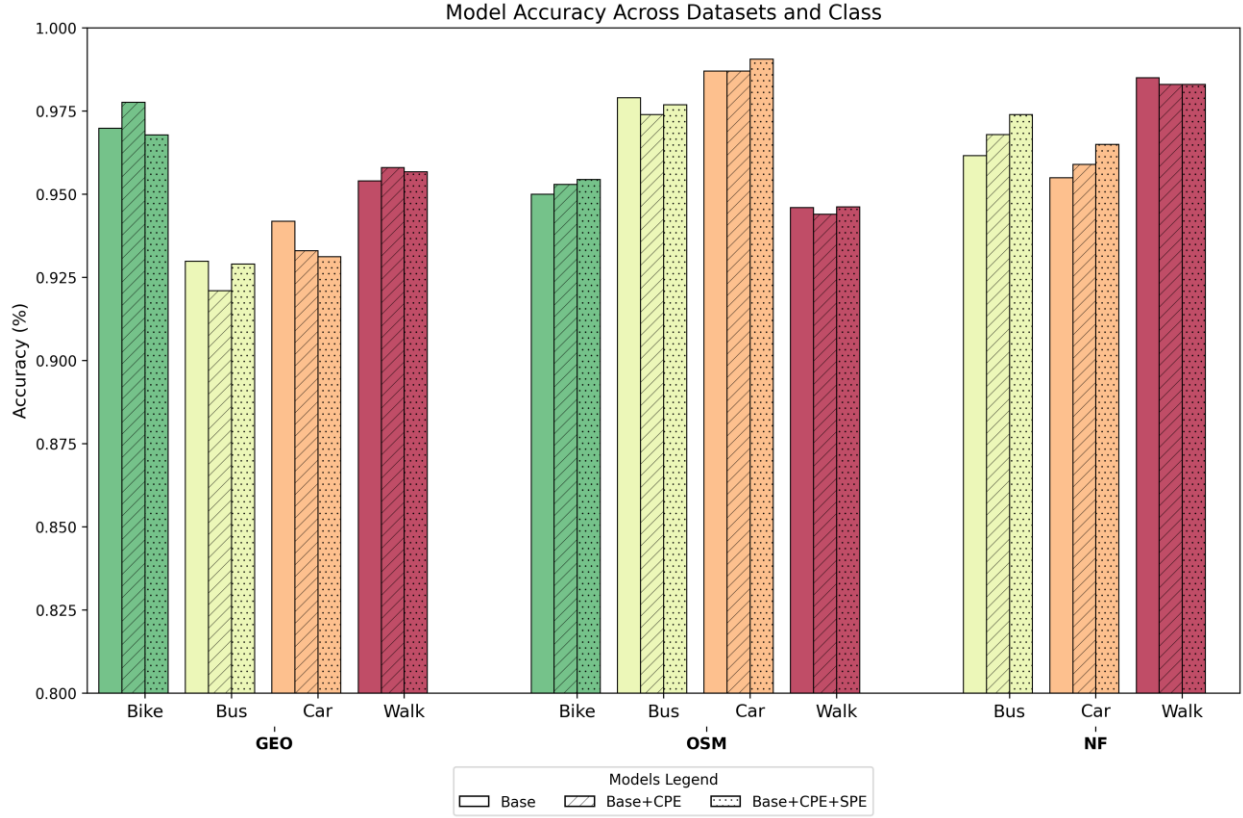


Figure 3 Model Accuracy Across Datasets and Class

Confusion Matrices

Finally, we examine the confusion matrices of our proposed transformer-based model. Each matrix summarizes how the model assigns predicted transportation modes to the true labels for each dataset.

For the GeoLife dataset, the main source of misclassification errors come between car and bus trips. A noticeable portion of bus trips are misclassified as cars and vice versa, but misclassification across the remaining classes is relatively limited. In contrast, for the OpenStreetMap dataset, the largest misclassification errors occur between the walking and biking classes, with a relatively small number of errors between motorized modes. One plausible explanation for this difference is the geographical areas of each dataset. GeoLife trajectories are primarily recorded in Beijing [13], a dense urban environment where buses and cars frequently share the same roads and traffic conditions, which may lead to similar speeds and spatiotemporal patterns, causing the misclassification errors between these two classes. By comparison, OpenStreetMap trajectories are drawn from a broader range of global settings. Car trips can occur on intercity roads or highways with distinct speed patterns, while walking and biking may often take place on shared paths or nature trails with similar leisurely speeds. A similar pattern is visible in the North Florida dataset, but with far fewer samples. In the North Florida dataset, misclassifications are rare, but still mostly occur between bus and car. While these interpretations are consistent with the observed error structure, they remain unverified. Additional analysis would be needed to confirm the extent to which geographical and environmental factors of the datasets drive these patterns.

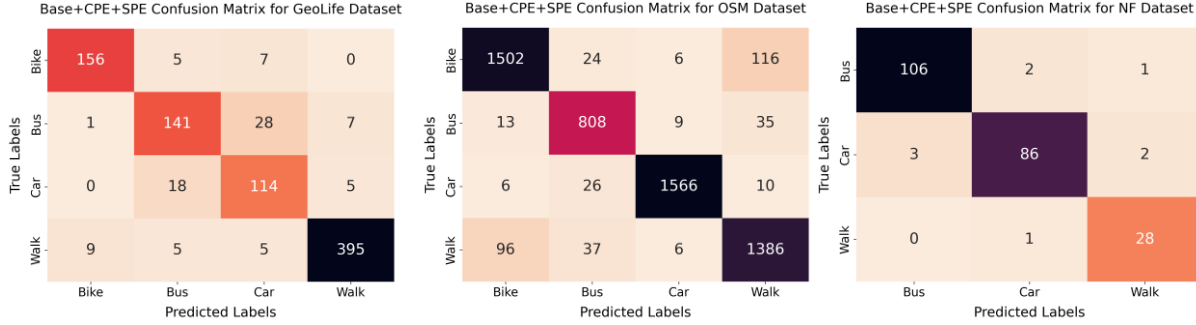


Figure 4 Confusion Matrices of Proposed Model Across Datasets

DISCUSSION

It is clear from the results of our ablation tests that the addition of spherical positional encoding offers a consistent improvement in performance, yielding 0.1 ~ 1.3% gains in test accuracy when compared to just using continuous point embedding. Our proposed transformer architecture that uses both continuous point embedding and spherical positional encoding performs the best out of all test models across almost all GPS trajectory datasets. This improvement highlights the importance of not only encoding the relative time and distance differences between GPS points, but also the absolute geographical locations of the GPS points. Continuous point embedding and spherical positional encoding complement each other due to their ability to inject information that the other strategy omits.

The usage of real-world contextual features also provides a modest gain in performance to almost all models. Its addition yielded 0.1 ~ 3.5% gains in test accuracy, providing the largest boost to models that struggled such as the vanilla RNN model. One possible explanation could be that models with limited ability to handle the spatial and temporal features of long trajectories may learn to instead rely strongly on the real-world contextual features to help differentiate trajectories. However, complex models such as LSTM, GRU and transformer-based models can more accurately model longer trajectories using only the spatial and temporal features. For these models, the real-world contextual features serve as assistance and provide a slight boost. Once again, this addition provides consistent improvements in model performance despite not being included in many existing literature on GPS trajectory mode classification [5,6,9]. Additionally, the data used to derive these real-world contextual features is simple and accessible. The global map data is hosted freely by the OpenStreetMap project, and there are several other third-party services that also freely host mirrored and region-specific versions of this data [14]. We suggest that all future research on GPS trajectory mode classification take advantage of this easy addition of real-world contextual information.

Another factor that had an impact on test performance was the data set used for training. Our models exhibited varying performance ranges across the datasets, with the lowest on GeoLife, then OpenStreetMap, and the highest performance on the North Florida dataset. The North Florida dataset experienced the highest performance, most likely due to the usage of three labels instead of four in addition to the trajectories being manually labeled. We expected performance on the OpenStreetMap dataset to be the lowest, due to its trajectories coming from a large range of geographical regions compared to the other datasets. Therefore, we were surprised to find that performance was lowest on GeoLife, which contained the smallest geographic region compared to the other datasets. A possible explanation is factors such as data quality may have a greater impact on model performance than geographic scope. This finding highlights a key issue for GPS trajectory mode classification, the lack of high-quality benchmark GPS trajectory datasets. High-quality benchmark datasets are rare, and the few existing ones are often not easily accessible. Many papers must curate their own custom datasets for model development, preventing a fair comparison against other models and reproducibility in work [5,6,7,8,9]. Additionally, complex models such as transformer-based architectures, require a substantial amount of data to achieve a strong performance. Even the widely used benchmark dataset, GeoLife, is relatively small for training complex models effectively. We believe that our usage of OpenStreetMap

GPS trajectories, which are free and accessible, highlights the potential for creating a large high-quality benchmark dataset.

CONCLUSIONS

With the ubiquity of mobile devices, GPS data emerges as a strong data source to glean insights into human transportation behavior. One crucial factor of transportation behavior is transportation mode, which has large impacts of transportation infrastructure and mobility efficiency. Consequently, classifying transportation mode from GPS trajectories can offer key insights to enable responsive and intelligent transportation systems. Transformer models offer a solution to accurately model the sequential aspects but fail to account for the spatial, temporal and geographical characteristics. Therefore, our paper address this research gap by designing a custom transformer model with novel techniques such as spherical positional encoding and real-world contextual features to solve the task of GPS trajectory mode classification. Our embedding strategy encodes the ordering, spatial, temporal and geographical relations between points which other papers have not achieved. Our novel strategy showcases consistent improvements compared to other transformer-based models across three GPS trajectory datasets. This boost in performance is a result of the complementary nature of CPE and SPE, which inject different information into the embeddings that the other technique omits. SPE captures the absolute geographical location of points, while CPE captures the relative time and distance differences between points. Our work also shows how the addition of real-world contextual features also provides a consistent boost in performance despite being rarely utilized in prior work.

These improvements in GPS trajectory mode classification have direct applications to the transportation industry. By improving upon the reliability and performance of these models, transportation institutions can properly extract travel behavior information in urban areas without relying on resource-intensive surveys and interviews. Urban planners can leverage this new information to make data-driven decisions on transportation infrastructure and policy development. Overall, our usage of custom positional encoding techniques and real-world contextual features in GPS trajectory mode classification contributes to building smarter, more efficacious transportation systems that can respond to the mobility needs of urban populations.

ACKNOWLEDGMENTS

The author acknowledges the use of AI tools in this submission. Specifically, OpenAI’s ChatGPT 4o model was utilized to assist in editing the paper and ensure the communication of ideas was clear and concise. However, all research and analysis presented in the paper is the original work of the authors.

AUTHOR CONTRIBUTIONS

The authors confirm contribution to the paper as follows: study conception and design: A. Wong, X. Yan; data collection: A. Wong; analysis and interpretation of results: A. Wong; draft manuscript preparation: A. Wong, X. Yan. All authors reviewed the results and approved the final version of the manuscript.

REFERENCES

1. Eluru, N., Chakour, V., & El-Geneidy, A. M. Travel mode choice and transit route choice behavior in Montreal: Insights from McGill University members commute patterns. *Public Transport*, 2012. 4(2): 129–149. <https://doi.org/10.1007/s12469-012-0056-2>
2. Dabiri, S., & Heaslip, K. Inferring transportation modes from GPS trajectories using a convolutional neural network. *Transportation Research Part C: Emerging Technologies*, 2018, 86: 360–371. <https://doi.org/10.1016/j.trc.2017.11.021>
3. Stopher, P., & Greaves, S. Household travel surveys: Where are we going? *Transportation Research Part A: Policy and Practice*, 2007. 41(5): 367–381. <https://doi.org/10.1016/j.tra.2006.09.005>
4. Bricka, S., Zmud, J., Wolf, J., & Freedman, J. Household travel surveys with GPS. *Transportation Research Record: Journal of the Transportation Research Board*, 2009. 2105(1): 51–56. <https://doi.org/10.3141/2105-07>
5. Liang, Y., Ouyang, K., Wang, Y., Liu, X., Chen, H., Zhang, J., Zheng, Y., & Zimmermann, R. Trajformer: Efficient trajectory classification with Transformers. *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022. 1229–1237. <https://doi.org/10.1145/3511808.3557481>
6. Burkhard, O., Becker, H., Weibel, R., & Axhausen, K. W. On the requirements on spatial accuracy and sampling rate for transport mode detection in view of a shift to passive signalling data. *Transportation Research Part C: Emerging Technologies*, 2020. 114: 99–117. <https://doi.org/10.1016/j.trc.2020.01.021>
7. Zhang, L., Darzi, A., Pan, Y., Yang, M., Sun, Q., Kabiri, A., Zhao, G., Ashoori, M., Namadi, S. S., Xiong, C., Jordan, G. W., Stewart, K., & Pack, M. L. Next Generation National Household Travel Survey National Origin-Destination Data: Passenger Origin-Destination Data Methodology Documentation. *Federal Highway Administration*, 2021
8. Tang, Q., & Cheng, H. Feature pyramid BILSTM: Using smartphone sensors for Transportation Mode Detection. *Transportation Research Interdisciplinary Perspectives*, 2024. 26. <https://doi.org/10.1016/j.trip.2024.101181>
9. Jiang, X., de, N., Pesaranghader, A., Hu, B., Silver, D., & Matwin, S. TrajectoryNet: An Embedded GPS Trajectory Representation for Point-based Classification Using Recurrent Neural Networks. *ArXiv (Cornell University)*, 2017. <https://doi.org/10.48550/arxiv.1705.02636>
10. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. Attention Is All You Need. *ArXiv*, 2017. <https://arxiv.org/abs/1706.03762>
11. Drosouli, I., Voulodimos, A., Mastorocostas, P., Miaoulis, G., & Ghazanfarpour, D. TMD-Bert: A Transformer-based model for transportation mode detection. *Electronics*, 2023. 12(3): 581. <https://doi.org/10.3390/electronics12030581>
12. Unlu, E. Spherical position encoding for Transformers. *ArXiv*, 2023. <https://doi.org/10.48550/arXiv.2310.04454>
13. Zheng, Y., Fu, H., Xie, X., Ma, W.-Y., & Li, Q. Geolife GPS trajectory dataset. *Microsoft Research*, 2011. <https://www.microsoft.com/en-us/research/publication/geolife-gps-trajectory-dataset-user-guide/>

- 1 14. Lyu, D., Liu, L., Campbell, C., Zhang, Y., & Yan, X. Potentials and limitations of large-scale,
2 individual-level mobile location data for food acquisition analysis. *ArXiv*, 2025.
3 <https://doi.org/10.48550/arXiv.2503.18119>
4
- 5 15. OpenStreetMap contributors. *OpenStreetMap*. <https://www.openstreetmap.org/>. Accessed July, 2025.